

Nombre:

DNI:

Examen final de teoría

Justifica todas tus respuestas. Una respuesta sin justificación se considerará errónea.

1. (1 punto) Definiciones

Define, brevemente, los siguientes conceptos:

a) Estructura union task_union

Estructura que identifica y caracteriza un proceso junto con su contexto de ejecución. Contiene el PCB del proceso junto con su pila de sistema.

b) Paso de parámetros a una llamada a sistema mediante registros

Procedimiento por el cual se pasan los parámetros de una llamada a sistema utilizando registros del procesador. El número de parámetros está limitado por el número de registros de la arquitectura.

c) Contexto software

Información mínima que necesita el sistema operativo, para restaurar el estado del proceso que se está ejecutando en modo usuario.

d) Paginación

Técnica de mapeo de rangos de direcciones lógicas del software, páginas lógicas, a direcciones físicas de memoria, páginas físicas o frames.

e) Reubicación dinámica

Reposicionamiento de un proceso en memoria para evitar la fragmentación de memoria en modelos de memoria donde la memoria física utilizada por un proceso tiene que ser, necesariamente, contigua.

f) Proceso paginador

Proceso del sistema encargado de hacer el mantenimiento de la memoria para optimizar el rendimiento del sistema operativo.

g) Portabilidad de código de usuario

Característica del código por el cual puede ser compilado y ejecutado sin modificación en diferentes sistemas operativos.

h) Pila de sistema

Estructura de memoria que guarda el contexto de ejecución del proceso actual cuando se está ejecutando en modo de máximos privilegios.

i) Excepción de fallo de página

Mecanismo del procesador para avisar al sistema operativo que no se puede traducir una dirección lógica a física.

j) Modo protegido

Modo de ejecución del procesador en el cual se activa la paginación y los niveles de privilegios de ejecución.

k) Compartición del espacio de direcciones físicas del kernel entre procesos

Mecanismo por el cual todos los procesos de un sistema operativo ven la misma imagen del núcleo del sistema operativo.

l) I-nodo

Estructura que identifica y caracteriza una tira de bytes en disco sin tipo.

m) Bloque de disco

Unidad mínima de transferencia del sistema operativo. Está compuesta por uno o más sectores.

n) Superbloque

Estructura al inicio de una partición de disco que contiene todos los metadatos sobre esta partición.

o) Asignación indexada de bloques de disco a ficheros

Método de asignación de bloques de datos de disco a ficheros por el cual se crea una lista encadenada de bloques de índices que contienen los índices de los bloques que contienen los datos del fichero.

p) Descriptor de dispositivo

Estructura en memoria que contiene toda la información sobre un controlador de dispositivo junto con los punteros a las funciones de éste.

q) Virtual File System

Capa del sistema de ficheros que abstrae y unifica las características de los sistemas de ficheros específicos con los cuales el sistema operativo es compatible.

r) Entrada/salida asíncrona

Método de entrada/salida a dispositivo en el cual se separa la petición de los datos de la recepción de estos. Para conseguir esto, la llamada al sistema de petición de datos no es bloqueante y se crea una segunda llamada al sistema para bloquear al thread en el caso de que los datos no estén disponibles.

s) Cambio de contexto

Mecanismo por el cual el sistema operativo cambia el proceso que actualmente se está ejecutando en la cpu.

t) Wrapper de la función de usuario del thread

Función de usuario que se ejecuta al crear un thread y que ejecuta la función de usuario establecida en la creación del thread junto con la llamada al sistema para terminar la ejecución de ese thread.

SOA (15/01/2026)

Nombre:

DNI:

2. (3 puntos) Memoria

El vector de PCBs del sistema operativo contiene todos los PCBs que se pueden utilizar para crear procesos. Como este vector es estático, el número de procesos concurrentes está limitado por el tamaño de este vector.

Para solucionar esta limitación, se proponen 2 posibles implementaciones:

- 1.- Modificar el tamaño de este vector de forma dinámica
- 2.- Eliminar este vector y reservar los PCBs de forma dinámica.

La gestión de los PCBs se hará dentro de las siguientes funciones:

```
// Devuelve un union task_union libre o (union task_union*)-1 si no hay ninguno disponible
union task_union* GetFreeTaskUnion();
```

```
// Marca un union task_union como libre
void FreeTaskUnion(union task_union *t_u);
```

Un union task_union ocupa una página lógica de 4KBs.

El resto de las estructuras de gestión de procesos no se modifican.

a) ¿En qué partes del código del sistema operativo se llamará a GetFreeTaskUnion?

Donde sea necesario crear un proceso. En ZeOS: `init_task1`, `init_idle`, `sys_fork`

b) ¿En qué partes del código del sistema operativo se llamará a FreeTaskUnion?

Cuando se necesite liberar un task_union. En ZeOS: en `sys_exit`.

Implementación 1:

Esta implementación se basa en que cuando se pida un nuevo union task_union, si no hay ninguno libre, se doblará la cantidad de memoria asignada al vector de PCBs. Cuando se libere algún union task_union, se comprobará si puede reducir la cantidad de memoria asociada al vector de PCBs, liberándola si es posible. Las reducciones de memoria siempre tienen que ser liberando la segunda mitad del vector de PCBs. La variable NPAGS contendrá siempre el número de páginas lógicas asociadas al vector de PCBs.

c) ¿De qué forma se comprueba, dentro de la función GetFreeTaskUnion, si existe algún union task_union disponible?

Mirando si la cola `free_queue` está o no vacía.

d) Indica los pasos que se tienen que hacer en caso de que no quede ningún union task_union disponible.

1.- Buscar un conjunto de 2xNPAGS páginas lógicas contiguas disponibles en la zona de la tabla de páginas correspondiente al sistema operativo.

2.- Asignar páginas físicas a la zona encontrada en 1

3.- Copiar el vector de PCBs a esa zona de la tabla de paginas

4.- Añadir los nuevos PCBs a la free_queue

5.- Reubicar todos los punteros a PCBs del sistema

e) Describe un caso en el cual existan 2xNPAGS páginas lógicas disponibles en la zona de sistema de la tabla de páginas y existan más de NPAGS físicas disponibles, pero no se pueda aumentar el tamaño del vector de PCBs

Cuando haya mucha fragmentación en la Tabla de páginas.

f) Indica, dentro de la función FreeTaskUnion, como se comprueba si se puede liberar parte del vector de PCBs.

Recorriendo las últimas NPAGS/2 posiciones para determinar si están o no en uso.

g) ¿Qué problemas presenta esta implementación? Describe un mínimo de 2 problemas

-El vector de PCBs se tiene que reubicar lo cual podría no ser posible.

-Fragmentación interna del vector de PCBs que evita que se pueda liberar memoria.

h) Plantea una solución para los problemas descritos anteriormente.

La solución es la implementación 2.

Implementación 2:

Cada vez que se necesite un nuevo task_union, se reservará una nueva página física que se mapeará en la zona de la tabla de páginas correspondiente al sistema operativo. Además, se añadirá el campo struct list_head siblings dentro del PCB. Cuando se cree un nuevo proceso, el task_union de éste nuevo proceso se encolará en el campo siblings del proceso actual. Cuando se libere un task_union, se liberará la página física asociada y el mapeo dentro de la tabla de páginas.

i) ¿Qué estructura del sistema operativo se puede eliminar con esta implementación?

La free_queue ya que los PCBs que existan en el sistema estarán todos en uso.

j) ¿Para qué sirve el campo siblings?

Para tener todos los PCBs de los procesos existentes en el sistema encadenados entre ellos.

k) ¿Se necesita alguna estructura extra para guardar todos los union task_unions que existen en el sistema en caso de que necesitemos recorrerlos?

No. Para esto existe el campo siblings.

l) ¿Soluciona esta implementación los problemas que la Implementación 1 presenta?

Si ya que al no necesitar que los PCBs estén consecutivos en memoria se elimina la fragmentación interna y no es necesario reubicar estos PCBs.

SOA (15/01/2026)

Nombre:

DNI:

3. (2 puntos) Sistema de ficheros

Dado un disco donde los sectores son de 512 Bytes, el sistema de ficheros tiene un tamaño de bloque de disco de 8 sectores, y el tamaño de un identificador de bloque son 4 bytes, queremos almacenar un fichero de 16 GBs.

Calcula el tamaño que ocupa en disco este fichero (sin contar bloques de disco de directorios ni I-nodes) con:

a) Asignación contigua

En asignación contigua no se necesita espacio extra de disco para guardar información de qué bloques de datos pertenecen al fichero. Por tanto, el fichero ocupará 16GBs

b) Asignación encadenada

Dentro de cada bloque de datos se tiene que almacenar el identificador (4 bytes) del bloque siguiente del fichero. El fichero ocupa $16\text{GB} / 4\text{KBs por bloque} = 4194304$ bloques de datos. Por tanto, los índices ocuparán $4194304 * 4 = 16777216$ Bytes más. Así que el espacio en disco que ocupará el fichero será $16\text{GBs} + 16\text{MBs}$.

c) Asignación indexada

El fichero ocupa $16\text{GB} / 4\text{KBs por bloque} = 4194304$ bloques de datos. Por tanto, los índices ocuparán $4194304 * 4 = 16777216$ Bytes más. Teniendo en cuenta que estos índices van en bloques separados, se necesitarán 4096 bloques para guardarlos. Estos bloques van encadenados entre ellos, por tanto, dentro de cada bloque de índices se tiene que guardar el identificador del siguiente bloque de índices, lo que añade 16 KBs más al espacio que ocupa el fichero en disco.

d) Asignación indexada multinivel

El fichero se compone de 4194304 bloques. Dentro del Inodo hay:

-10 índices directos: no ocupan espacio extra. Quedan 4194294 bloques a indentar.

-1 índice indirecto: se añade un bloque de índices para los siguientes 1024 bloques. Se añaden 4KB al espacio ocupado en disco.

-1 doblemente indirecto: se añade 1 primer bloque de índices que apunte a 1024 bloques de índices. Cubre los siguientes 1024x1024 bloques. Quedan 3144964 bloques a indentar. Se añaden $4\text{KB} + 1024 * 4\text{KB}$.

-1 triplemente indirecto: se necesitan, de tercer nivel, $(3144965/1024) = 3070$ bloques de índices. Para indexar estos 3070 bloques se necesitan, de segundo nivel, $(3070/1024) = 3$ bloques. Y para indexar estos 3 bloques de segundo nivel, solo se necesita 1 bloque de primer nivel. Por tanto, se añaden $4\text{KBs} + 3 * 4\text{KBs} + 3070 * 4\text{KBs}$.

e) Calcula, en el caso de que la asignación usada sea encadenada en tabla, el tamaño de la FAT en el caso de que el disco sea de N GBs.

$(N\text{GBs} / 4\text{KB por bloque}) * 4\text{ Bytes por índice}$.

4. (2 puntos) Procesos

Actualmente, la llamada al sistema fork, ejecuta las siguientes acciones:

- 1.-Asignar PID del hijo
- 2.-Asignar PCB del hijo
- 3.-Copiar contexto de ejecución del padre al hijo
- 4.-Asignar memoria física al proceso hijo para crear su imagen
- 5.-Asignar, de forma temporal, la memoria física del proceso hijo al padre
- 6.-Copiar el contenido de la memoria del proceso padre a esta memoria asignada
- 7.-Inicializar las estructuras necesarias del proceso hijo
- 8.-Crear contexto de ejecución del proceso hijo
- 9.-Encolar PCB del hijo en la cola de ready
- 10.-Return PID del hijo

Queremos modificar esta implementación para minimizar el tiempo que el proceso padre está ejecutando el fork. Para ello, hacemos que la mayoría de las acciones se ejecuten cuando el proceso hijo vaya a pasar por primera vez de Ready a Run. Así, las acciones 1, 2, 3, 8, 9 y 10 las realizará el padre y el resto las ejecutará el hijo.

- a) Con esta distribución de trabajo entre padre e hijo, ¿Se reduce el tiempo de ejecución de fork en el proceso padre?

Si ya que la parte más costosa de `sys_fork` es la copia de memoria de padre a hijo.

- b) ¿En qué parte del código, visto en clase, implementarías las acciones que tiene que realizar el hijo al pasar a ready?

En `ret_from_fork` ya que la copia de memoria se tiene que realizar la primera vez que el proceso hijo pase a RUN.

- c) Cuando ejecutamos el siguiente código, vemos que, a veces, la variable "a" tiene el valor 1 en el hijo y otras no. ¿Cómo es posible?

```
int main()
{
    int a=0;
    switch (fork()) {
        case 0: while(1);
        default: a=1;
    }
    while(1);
}
```

Se debe a que hay una condición de carrera. Si después de que se cree el proceso hijo se sigue ejecutando el padre, éste podrá modificar posiciones de memoria. Estas modificaciones se copiarán cuando el hijo pase por primera vez a RUN. Por tanto, el orden de ejecución de los procesos hace que el resultado varíe.

- d) Sin modificar la distribución de trabajo que hemos hecho en el fork, ¿cómo solucionarías el problema del apartado anterior?

Haciendo que al finalizar `sys_fork` el que se ejecute sea el proceso hijo y el padre se encole en la cola de Ready.

Nombre:

DNI:

5. (2 puntos) Gestores

Disponemos en nuestro sistema de un sistema de ficheros virtual que abstrae dos sistemas de ficheros: EXT4 y EXT2. Para ello, tanto el sistema de ficheros virtual como los sistemas de ficheros EXT4 y EXT2 disponen, cada uno de ellos, de un gestor que realiza las operaciones de entrada/salida. El gestor del sistema de ficheros virtual (VFS) traduce las peticiones que se realizan a peticiones a los gestores de EXT4 y EXT2 que son los que acaban accediendo a las particiones. Los gestores de EXT4 y EXT2 pueden trabajar concurrentemente. El gestor de EXT4 atiende las peticiones de forma secuencial, mientras que el gestor de EXT2 puede procesar más de una petición a la vez (es multiprogramado). Los gestores de EXT4 y EXT2 solamente son accesibles a través del gestor de VFS.

a) ¿Qué estructuras de datos se necesitan para poder enviar peticiones al gestor de VFS?

La tabla de canales, tabla de ficheros abiertos, tabla de Inodes (si existe), descriptor de dispositivos, cola de IORBs

b) ¿Dentro de qué función(es) se le envían peticiones al gestor de VFS?

Dentro de las funciones sys_* que son independientes del dispositivo.

c) ¿Qué estructuras de datos se necesitan para poder enviar peticiones entre el gestor VFS y los gestores EXT4 y EXT2?

Las colas de IORBs en las que el VFS encolarà las peticiones para EXT2/EXT4.

d) ¿Cuántos semáforos se necesitan para las peticiones entre el gestor de VFS y los gestores de EXT4 y EXT2?

1 de sincronización por gestor, 1 de exclusión mutua por cola de IORBs y 1 de sincronización por petición.

e) Escribe el pseudo-código del gestor de EXT4

```
while (1)
{
    Sem_wait(&sem_gestor);
    Recoger_peticion()
    Do I/O
    Encolar_resultados()
    Sem_post(&peticion);
}
```

La partición EXT4 está alojada en un disco duro que acepta hasta 16 lecturas/escrituras en paralelo, mientras que la partición de EXT2 se encuentra en un disco duro antiguo que solo acepta 1 acceso de lectura/escritura.

Indica, para los siguientes escenarios, el número de threads óptimo que debe tener el VFS para paralelizar, al máximo, los accesos a esos discos.

f) La entrada/salida es siempre síncrona

Dado que EXT4 solo puede procesar una petición y que el disco donde se encuentra EXT2 solo puede hacer un acceso concurrente, con 2 threads en VFS será suficiente.

g) La entrada/salida es asíncrona

Dado que ahora se puede paralelizar el acceso al disco de EXT2 que puede realizar un máximo de 16 accesos simultáneos, será suficiente con 17 threads (1 para EXT4 y 16 para EXT2)

Información del Sistema Operativo

Cada proceso contiene una estructura para guardar su espacio de direcciones, consistente en un directorio de páginas con una única entrada de válida que es su tabla de páginas. Cada una de las entradas de la tabla de páginas usada por la MMU (*page_table_entry*) contiene, entre otros, los siguientes campos (codificados en una estructura de 32 bits):

- **present**: *Present flag*, si este bit està a 1, la pàgina està a memòria principal; si està a 0, la pàgina no està a memòria principal i llavors la resta de bits de l'entrada es poden usar pel sistema operatiu.
- **pbase_addr**: *Address field*, camp amb els 20 bits més significatius de l'adreça física d'un marc de pàgina (frame).
- **user**: *User/Supervisor flag*, bit per indicar si la pàgina és d'usuari o sistema
- **rw**: *Read/Write flag*, bit per indicar els permisos d'accés de la pàgina (*Read/Write* o *Read*)

El sistema también dispone de las siguientes rutinas:

- **struct task_struct *current()**: Retorna la *task_struct* del proceso actual.
- **int alloc_frame()**: Reserva un frame de memoria física.
- **void free_frame (int frame)**: Libera un frame de memoria.
- **page_table_entry * get_PT(struct task_struct *t)**: Retorna la tabla de paginas del proceso *t*.
- **page_table_entry * get_DIR(struct task_struct *t)**: Retorna el directorio de páginas del proceso *t*.
- **set_CR3 (page_table_entry * dir)**: Sobreescribe el registro CR3 con el nuevo directorio de páginas *dir*, provoca una invalidación de la TLB.
- **int get_frame (page_table_entry *pt, int logical_page)**: Retorna el frame asignado a una página lógica en la tabla de páginas de un proceso determinado.
- **int set_ss_page (page_table_entry *pt, int logical_page, int frame)**: Asigna un *frame* a una página lógica de la tabla de paginas *pt*.
- **int del_ss_page (page_table_entry *pt, int logical_page)**: Borra una entrada de la tabla de páginas.